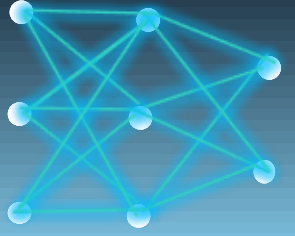




Deep Learning

Dr. Shay Maymon



Evaluation of Gradients

- Numerical Differentiation:**

Forward Difference

$$\frac{\partial L_n}{\partial w_{ij}} \approx \frac{L_n(w_{ij} + \varepsilon) - L_n(w_{ij})}{\varepsilon}$$

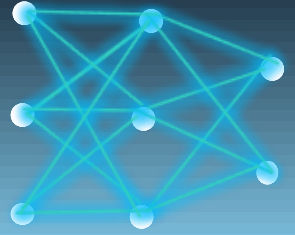
Backward Difference

$$\frac{\partial L_n}{\partial w_{ij}} \approx \frac{L_n(w_{ij}) - L_n(w_{ij} - \varepsilon)}{\varepsilon}$$

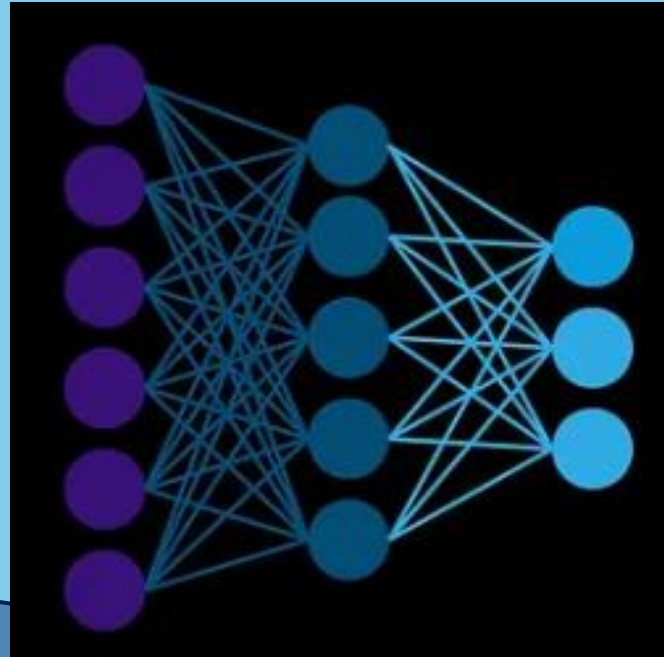
Centered Difference

$$\frac{\partial L_n}{\partial w_{ij}} \approx \frac{L_n(w_{ij} + \varepsilon) - L_n(w_{ij} - \varepsilon)}{2\varepsilon}$$

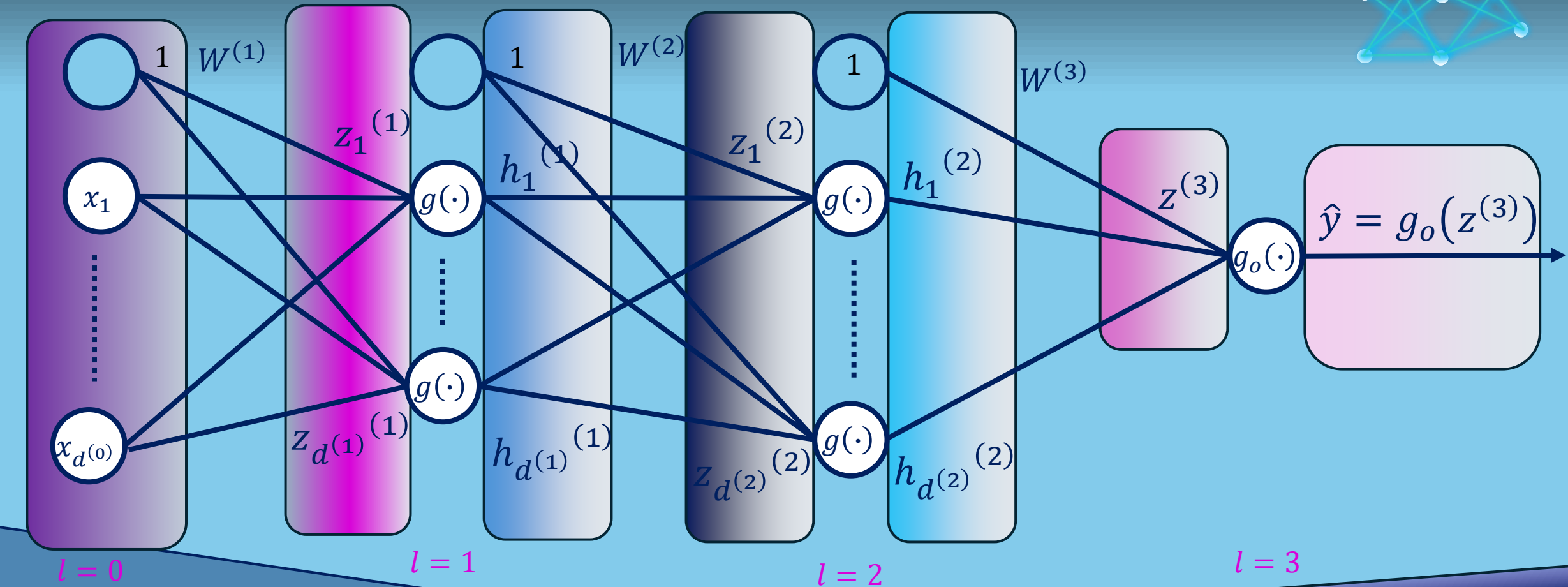
Deep FeedForward Network



- There are L layers labeled $l = 0, 1, 2, \dots, L$ ($l = 0$ input $l = L$ Output)
- Each layer l has 'dimension' $d^{(l)} - d^{(l)} + 1$ nodes (bias node labeled 0)



Forward Propagation



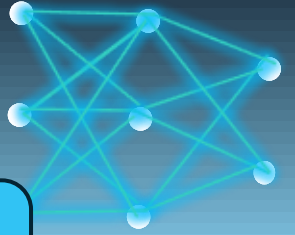
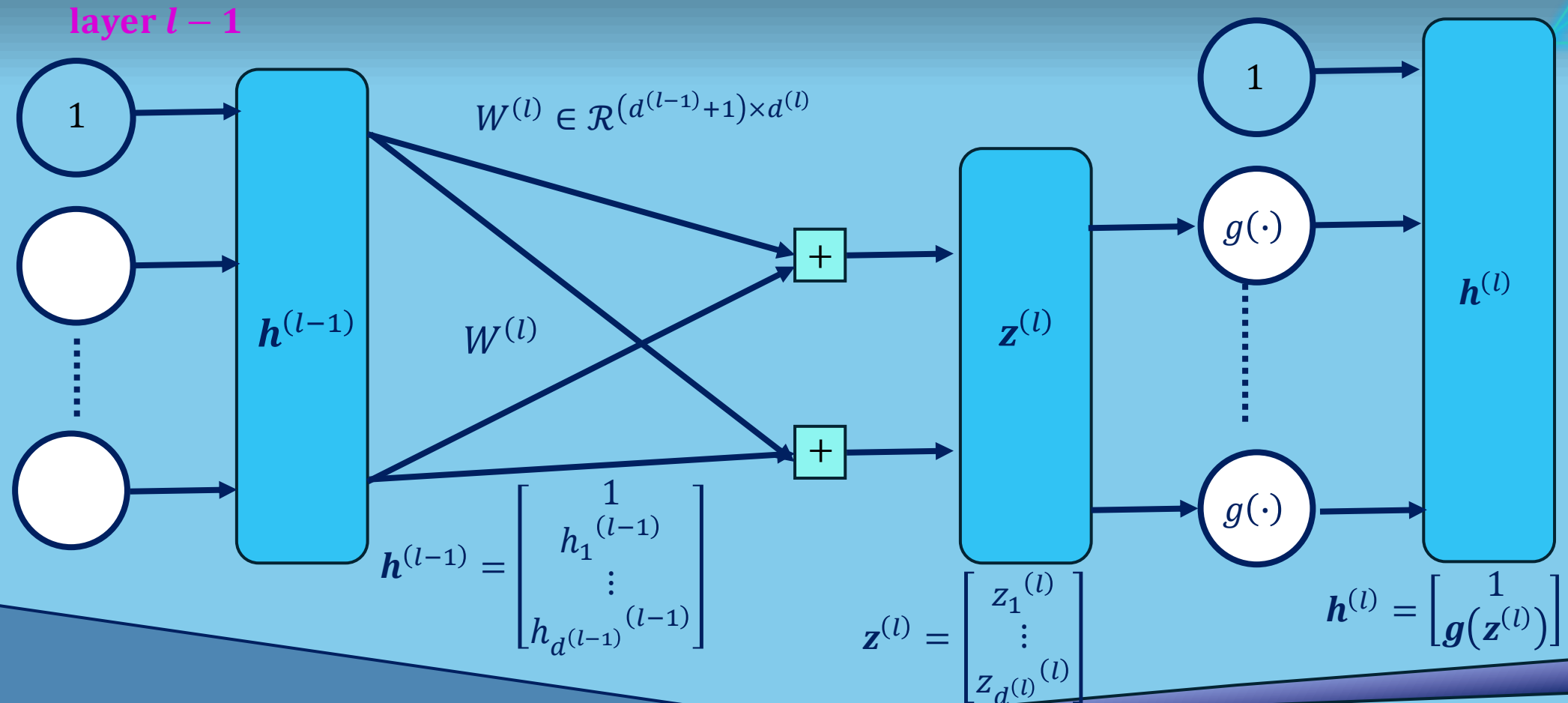
Input layer

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)T} \mathbf{h}^{(l-1)}$$

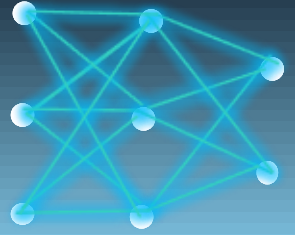
$$\mathbf{h}^{(l)} = \begin{bmatrix} 1 \\ g(\mathbf{z}^{(l)}) \end{bmatrix}$$

Output layer

Forward Propagation

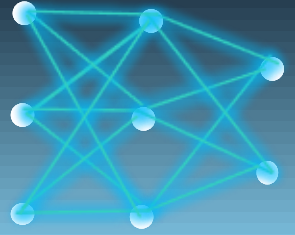


Forward Propagation



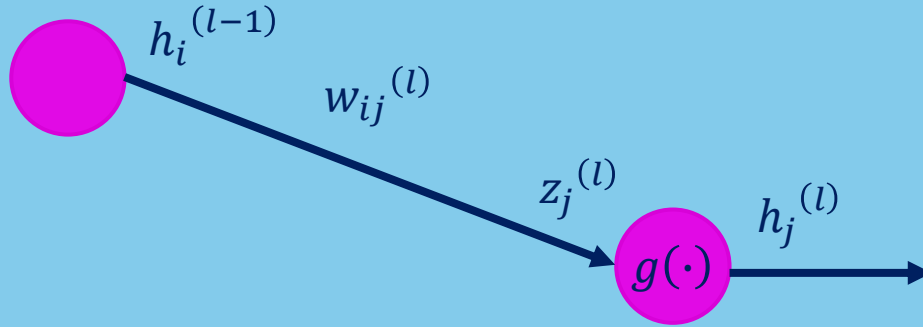
- $\mathbf{h}^{(0)} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix}$ [Initialization]
- **for** $l = 1$ to L **do** [Forward Propagation]
 - ✓ $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{h}^{(l-1)}$
 - ✓ $\mathbf{h}^{(l)} = \begin{bmatrix} 1 \\ g(\mathbf{z}^{(l)}) \end{bmatrix}$
- $\hat{\mathbf{y}} = \mathbf{h}^{(L)}$ [Output]

Backpropagation algorithm



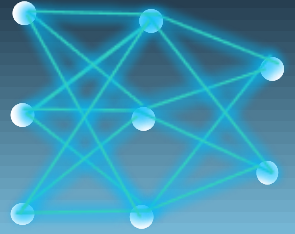
$$J(w) = \frac{1}{N} \sum_{n=1}^N e_n$$

$$\frac{\partial e}{\partial w_{ij}^{(l)}} = \frac{\partial e}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot h_i^{(l-1)}$$



$$z_j^{(l)} = \sum_{k=0}^{d^{(l-1)}} w_{kj}^{(l)} h_k^{(l-1)}$$

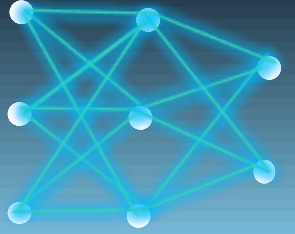
Backpropagation algorithm



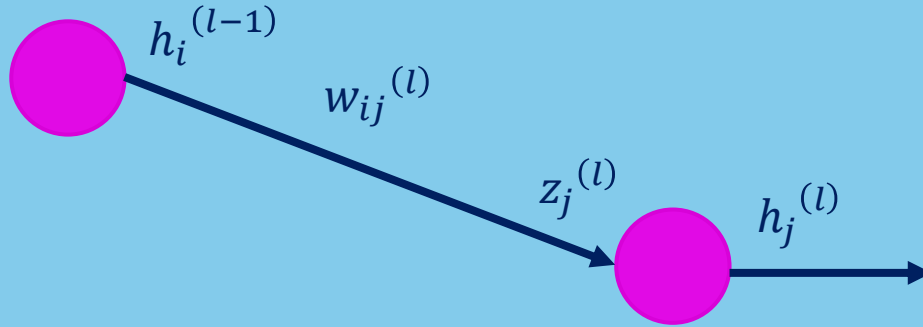
$$\frac{\partial e}{\partial w_{ij}^{(l)}} = \frac{\partial e}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot h_i^{(l-1)}$$

$$\frac{\partial e}{\partial W^{(l)}} = \mathbf{h}^{(l-1)} \boldsymbol{\delta}^{(l)T} \quad W^{(l)} \in \mathcal{R}^{(d^{(l-1)}+1) \times d^{(l)}}$$

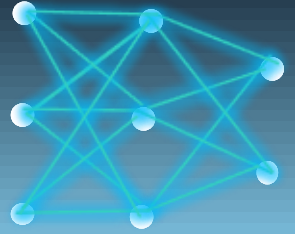
Backpropagation algorithm



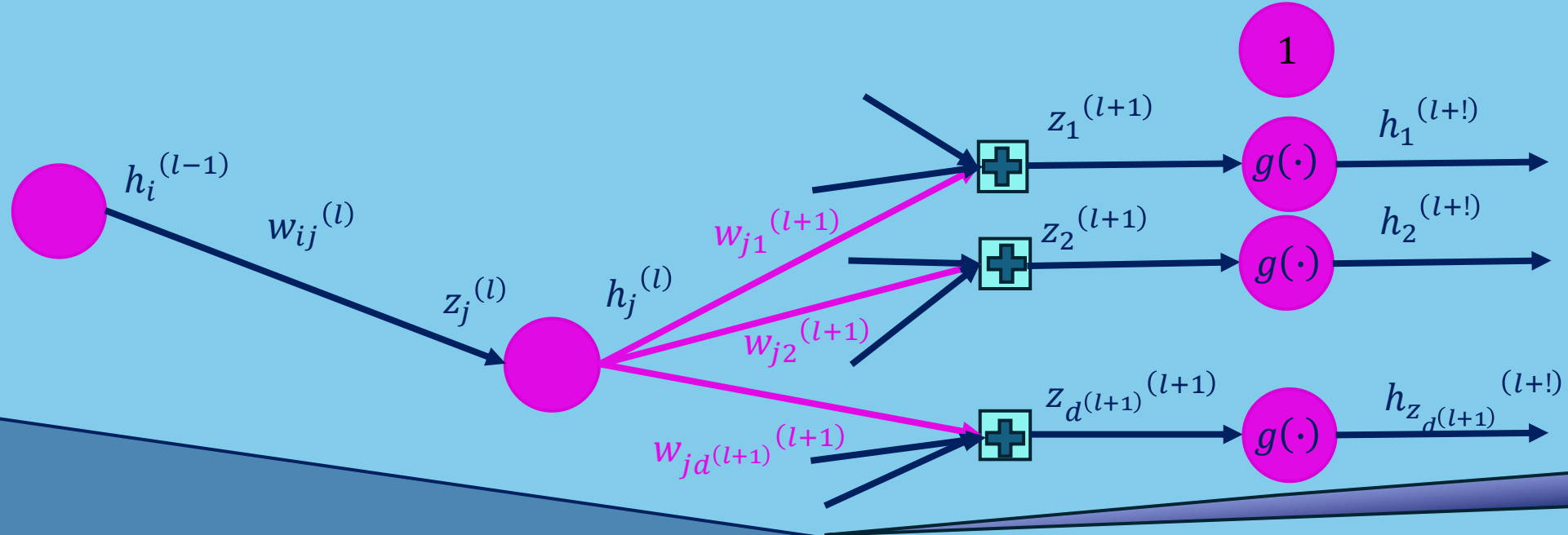
$$\delta_j^{(l)} = \frac{\partial e}{\partial z_j^{(l)}} = \frac{\partial e}{\partial h_j^{(l)}} \cdot \frac{\partial h_j^{(l)}}{\partial z_j^{(l)}} = \frac{\partial e}{\partial h_j^{(l)}} \cdot g'(z_j^{(l)})$$



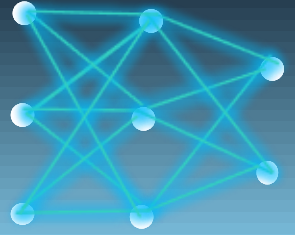
Backpropagation algorithm



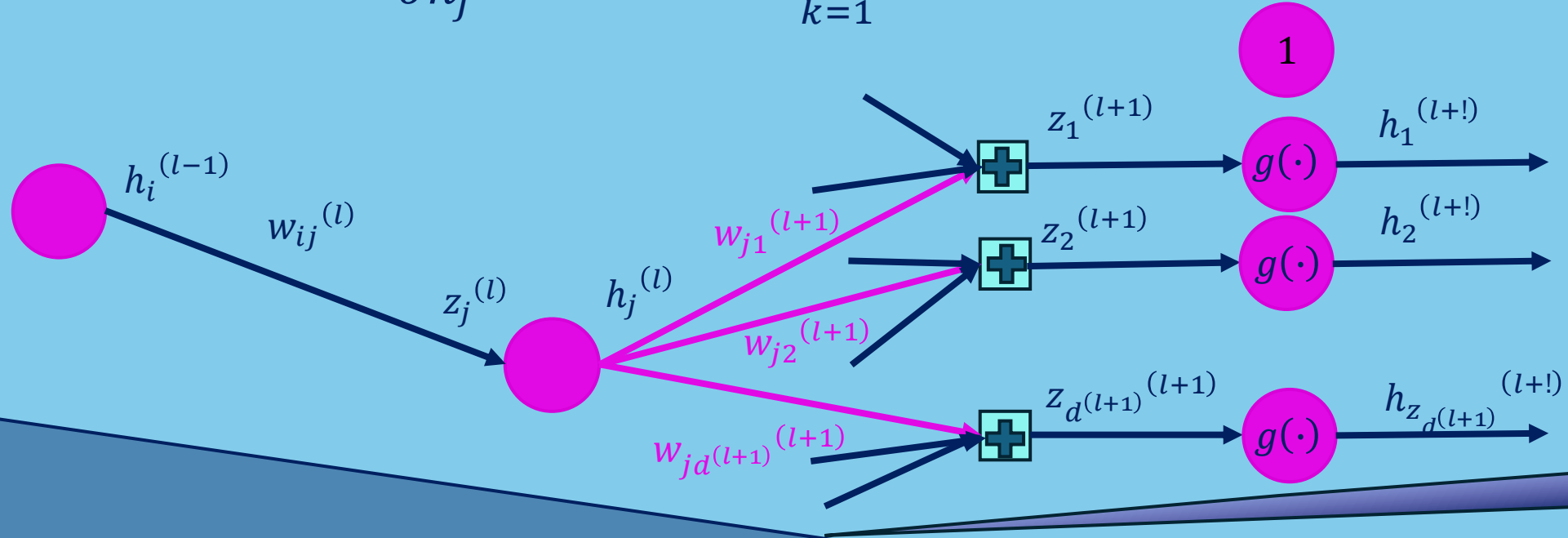
$$\frac{\partial e}{\partial h_j^{(l)}} = \sum_{k=1}^{d^{(l+1)}} \frac{\partial e}{\partial z_k^{(l+1)}} \cdot \frac{\partial z_k^{(l+1)}}{\partial h_j^{(l)}} = \sum_{k=1}^{d^{(l+1)}} \delta_k^{(l+1)} \cdot w_{jk}^{(l+1)}$$



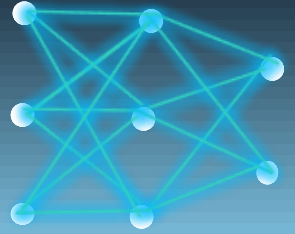
Backpropagation algorithm



$$\delta_j^{(l)} = g'(z_j^{(l)}) \cdot \frac{\partial e}{\partial h_j^{(l)}} = g'(z_j^{(l)}) \sum_{k=1}^{d^{(l+1)}} \delta_k^{(l+1)} \cdot w_{jk}^{(l+1)}$$



Backpropagation algorithm

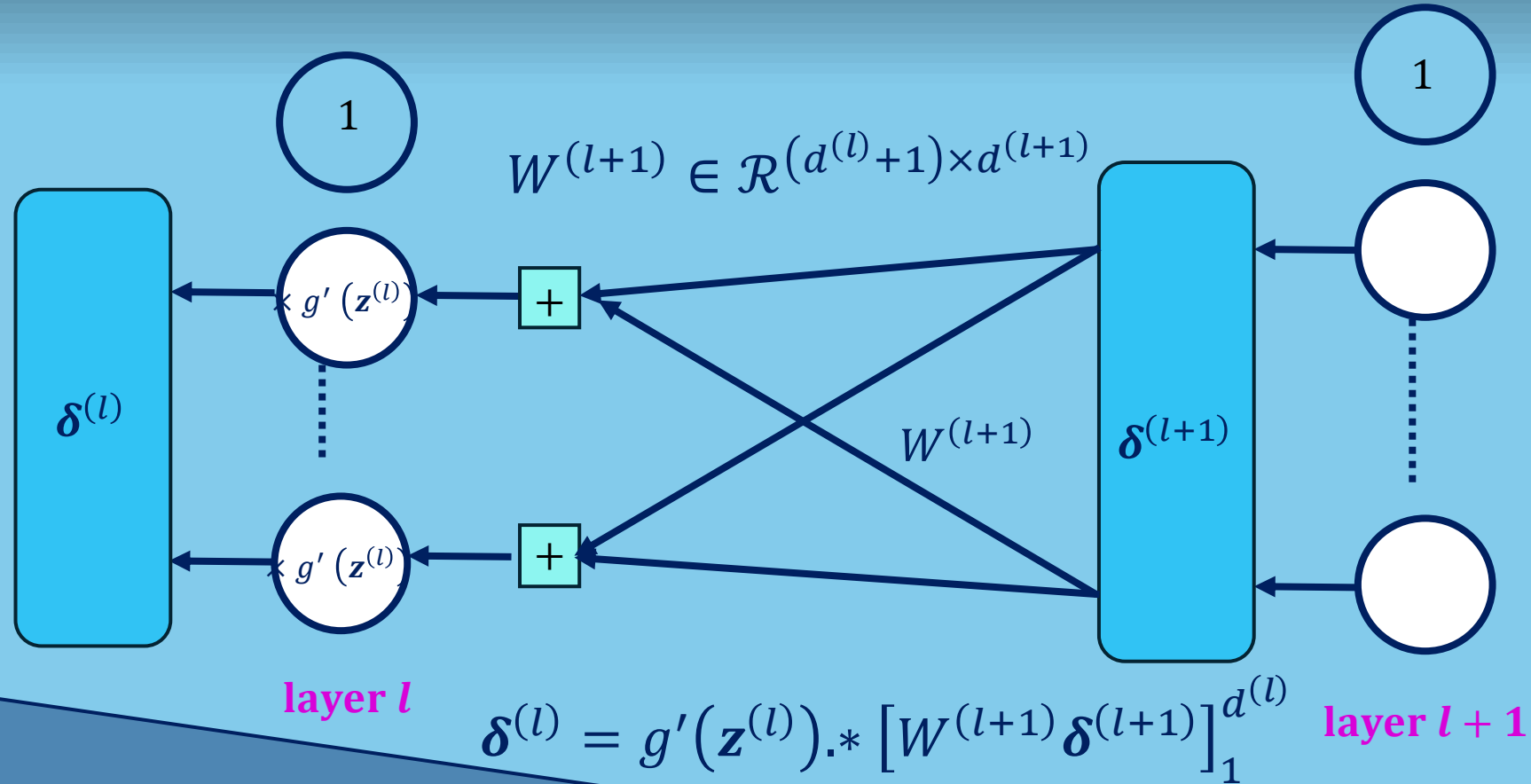
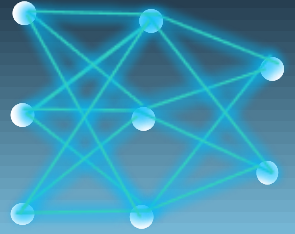


$$\delta_j^{(l)} = g'(z_j^{(l)}) \sum_{k=1}^{d^{(l+1)}} \delta_k^{(l+1)} \cdot w_{jk}^{(l+1)}, j = 1, 2, \dots, d^{(l)}$$

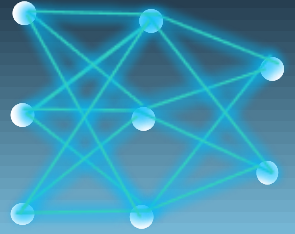
$$\boldsymbol{\delta}^{(l)} = g'(\mathbf{z}^{(l)}) .* [\mathbf{W}^{(l+1)} \boldsymbol{\delta}^{(l+1)}]_1^{d^{(l)}}$$

$$\mathbf{W}^{(l+1)} \in \mathcal{R}^{(d^{(l)}+1) \times d^{(l+1)}}$$

Backpropagation



Backpropagation



- **Input:** a data point (x, y)
- Run **Forward propagation** on x to compute and save:

$$z^{(l)} \text{ for } l = 1, 2, \dots, L$$

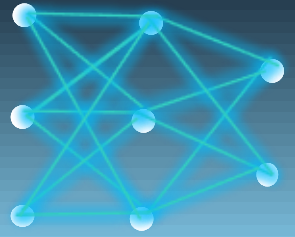
$$h^{(l)} \text{ for } l = 0, 1, \dots, L$$

- $\delta^{(L)} = \frac{\partial e}{\partial z^{(L)}}$ **[Initialization]**

- **for** $l = L - 1$ **to** 1 **do** **[Backpropagation]**

$$\checkmark \delta^{(l)} = g'(z^{(l)}) .* [W^{(l+1)} \delta^{(l+1)}]_1^{d^{(l)}}$$

Algorithm to Compute $J(\mathbf{w})$ and $\mathbf{g}$

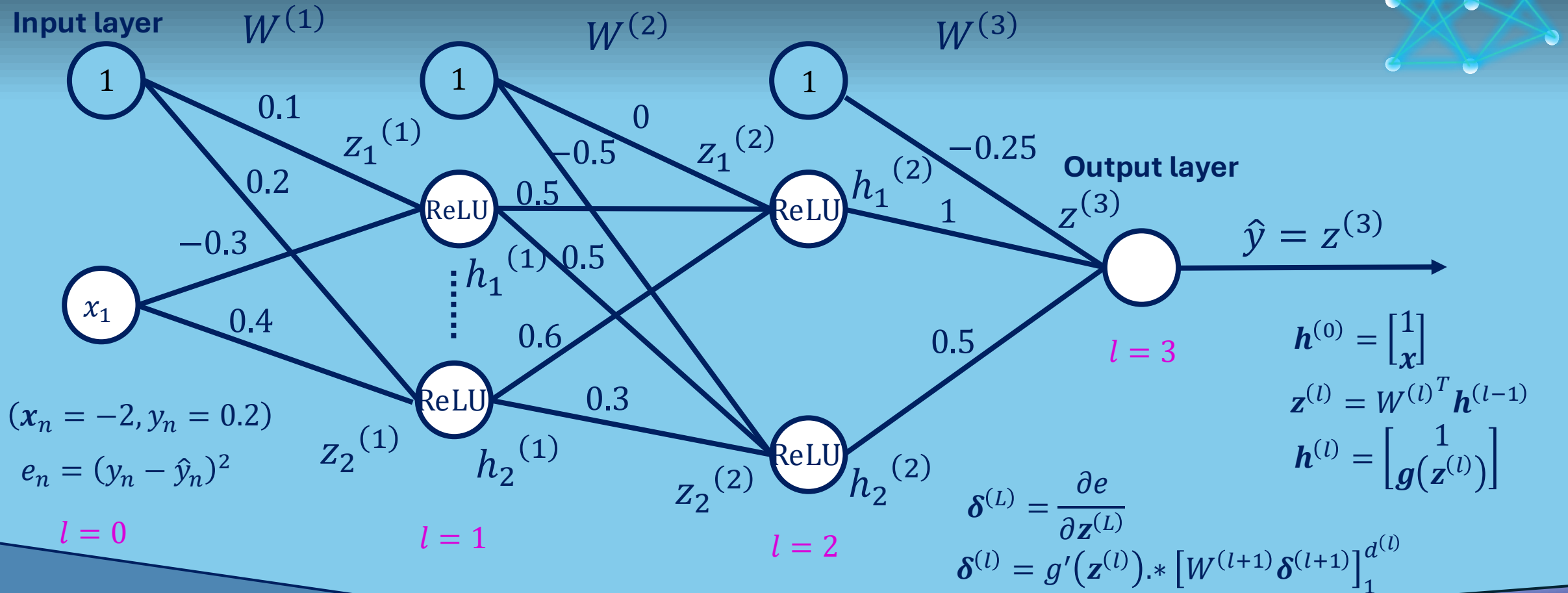


- **Input:** $\mathbf{w} = \{W^{(1)}, W^{(2)}, \dots, W^{(L)}\}$, $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$
- **Output:** error $J(\mathbf{w})$ and gradient $\mathbf{g} = \{G^{(1)}, G^{(2)}, \dots, G^{(L)}\}$
- $J(\mathbf{w}) = 0, G^{(l)} = 0, \{W^{(1)}, W^{(2)}, \dots, W^{(L)}\}$ [Initialization]
- **for each data point** $(\mathbf{x}_n, y_n), n = 1, 2, \dots, N$ **do**
 - Compute $\mathbf{h}^{(l)}$ for $l = 0, 1, \dots, L$ and $\mathbf{z}^{(l)}$ for $l = 1, 2, \dots, L$ [Forward propagation]
 - Compute $\delta^{(l)}$ for $l = L, \dots, 1$ [Backpropagation]
 - $J(\mathbf{w}) = J(\mathbf{w}) + \frac{1}{N} e_n(\mathbf{w})$
 - **for** $l = 1, \dots, L$ **do**
 - $G^{(l)}(\mathbf{x}_n) = \mathbf{h}^{(l-1)} \delta^{(l)T}$
 - $G^{(l)} = G^{(l)} + \frac{1}{N} G^{(l)}(\mathbf{x}_n)$

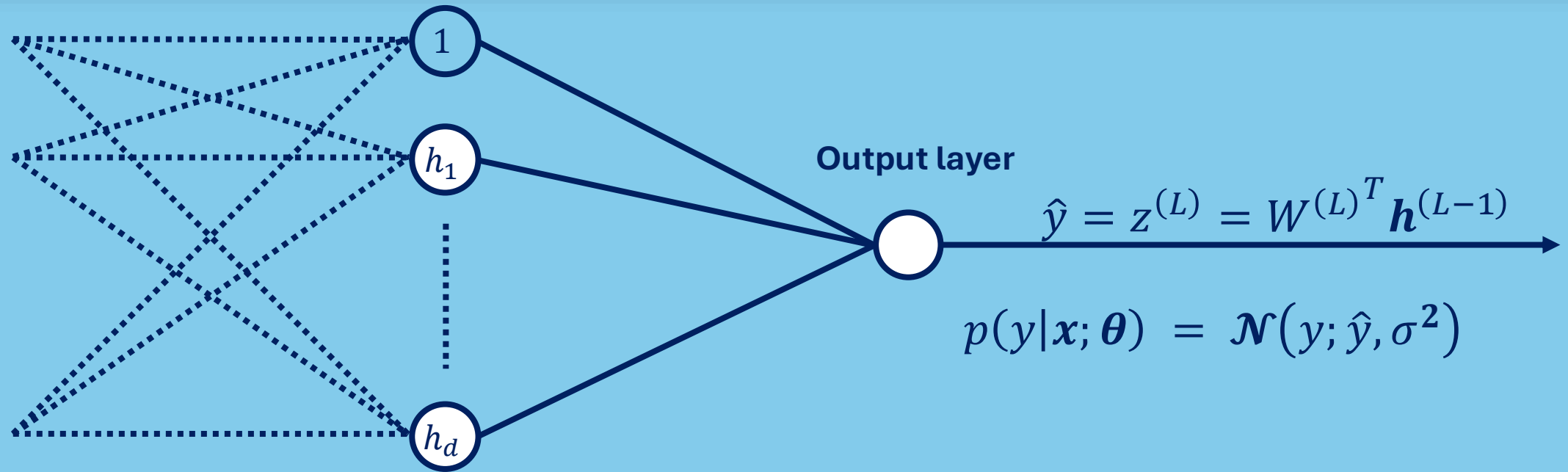
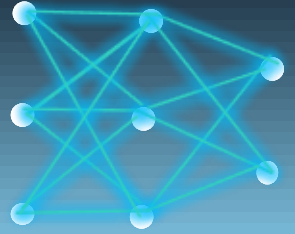
$$J(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N e_n$$

$$G^{(l)} = \frac{\partial J(\mathbf{w})}{\partial W^{(l)}} = \frac{1}{N} \sum_{n=1}^N \frac{\partial e_n}{\partial W^{(l)}} = \frac{1}{N} \sum_{n=1}^N G^{(l)}(\mathbf{x}_n)$$

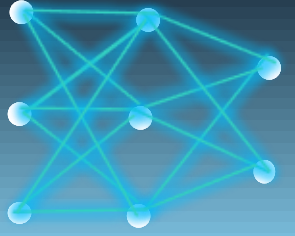
Example



Linear units for Gaussian Output distribution



Linear Units for Gaussian Output distribution



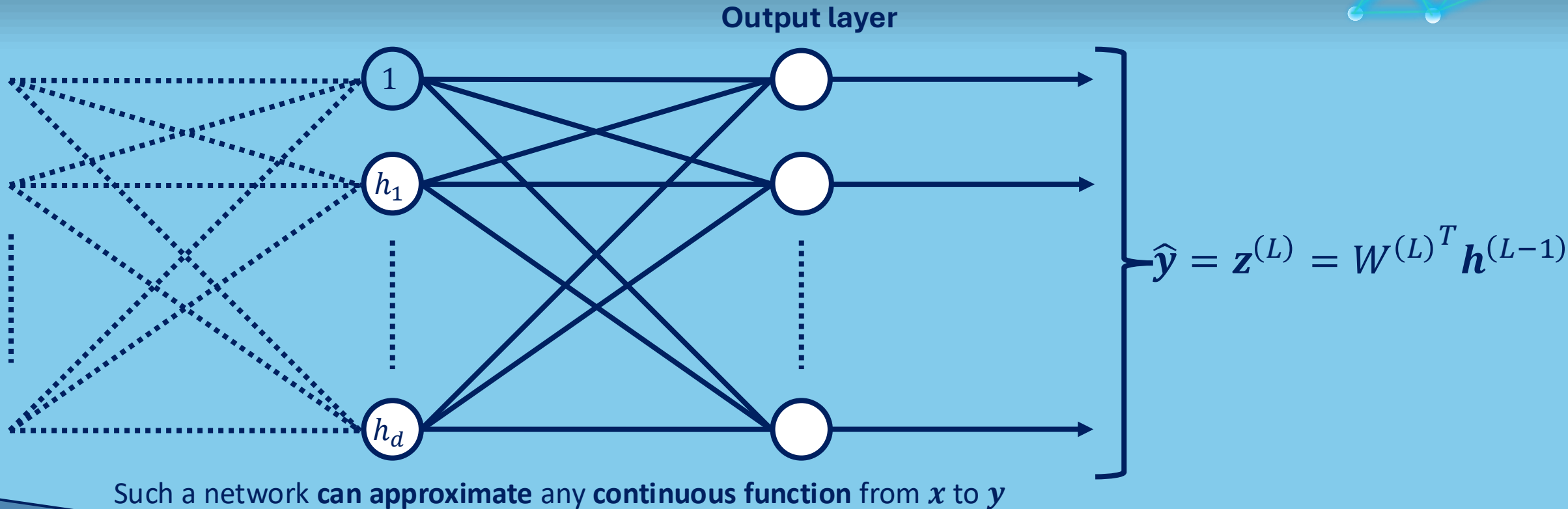
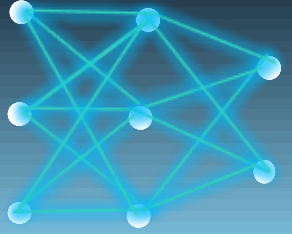
Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ i.i.d.

$$p(y_n | \mathbf{x}_n; \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2}$$

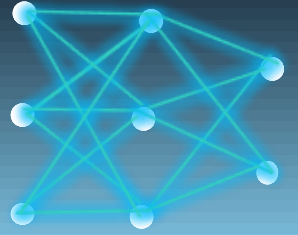
$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N (y_n - f(\mathbf{x}_n; \boldsymbol{\theta}))^2$$

$$\delta^{(L)} = \frac{\partial e}{\partial \mathbf{z}^{(L)}} = ?$$

Linear units for Gaussian Output distribution



Linear Units for Gaussian Output distribution



$$\hat{\mathbf{y}} = \mathbf{z}^{(L)} = \mathbf{W}^{(L)T} \mathbf{h}^{(L-1)}$$

$$p(\mathbf{y}|\mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y}; \hat{\mathbf{y}}, \sigma^2 \mathbf{I})$$

Training Data: $\{(\mathbf{x}_n, \mathbf{y}_n)_{n=1}^N\}$ i.i.d.

$$J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{n=1}^N \|\mathbf{y}_n - \hat{\mathbf{y}}_n\|^2$$

$$\boldsymbol{\delta}^{(L)} = \frac{\partial e}{\partial \mathbf{z}^{(L)}} = ?$$

Binary Classification

$$f(\mathbf{x}; \boldsymbol{\theta}) = \sigma \left(\underbrace{\mathbf{w}^T \mathbf{h} + b}_{z} \right)$$

Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ i.i.d.

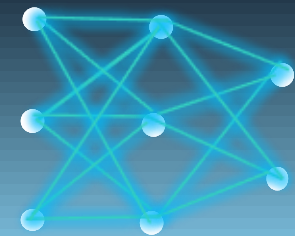
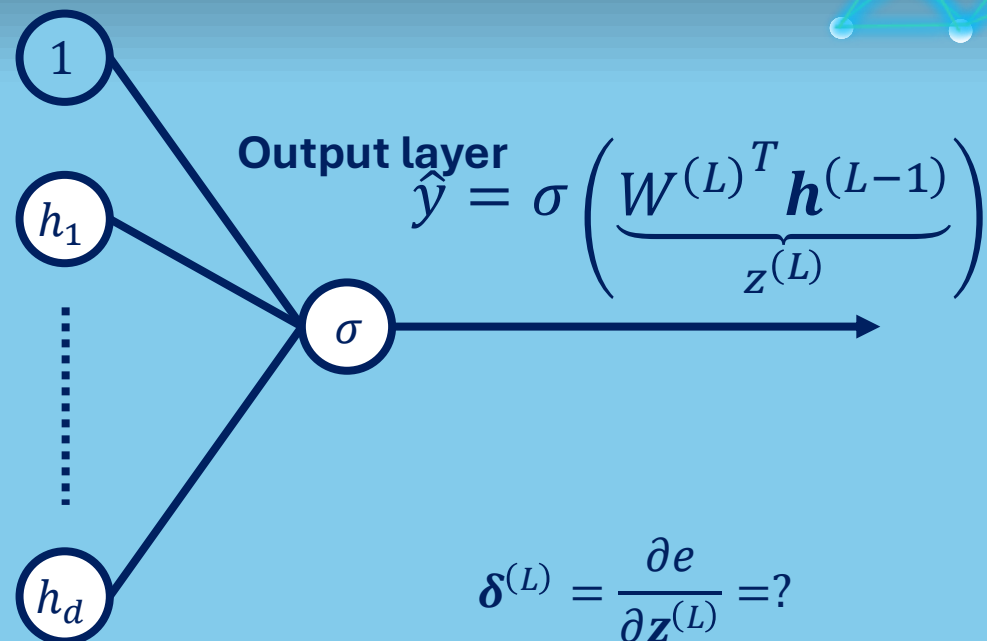
$$p(y_n | \mathbf{x}_n; \boldsymbol{\theta}) = f(\mathbf{x}_n; \boldsymbol{\theta})^{y_n} \cdot (1 - f(\mathbf{x}_n; \boldsymbol{\theta}))^{1-y_n}$$

$$J(\boldsymbol{\theta}) = - \sum_{n=1}^N y_n \log(f(\mathbf{x}_n; \boldsymbol{\theta})) + (1 - y_n) \log(1 - f(\mathbf{x}_n; \boldsymbol{\theta}))$$

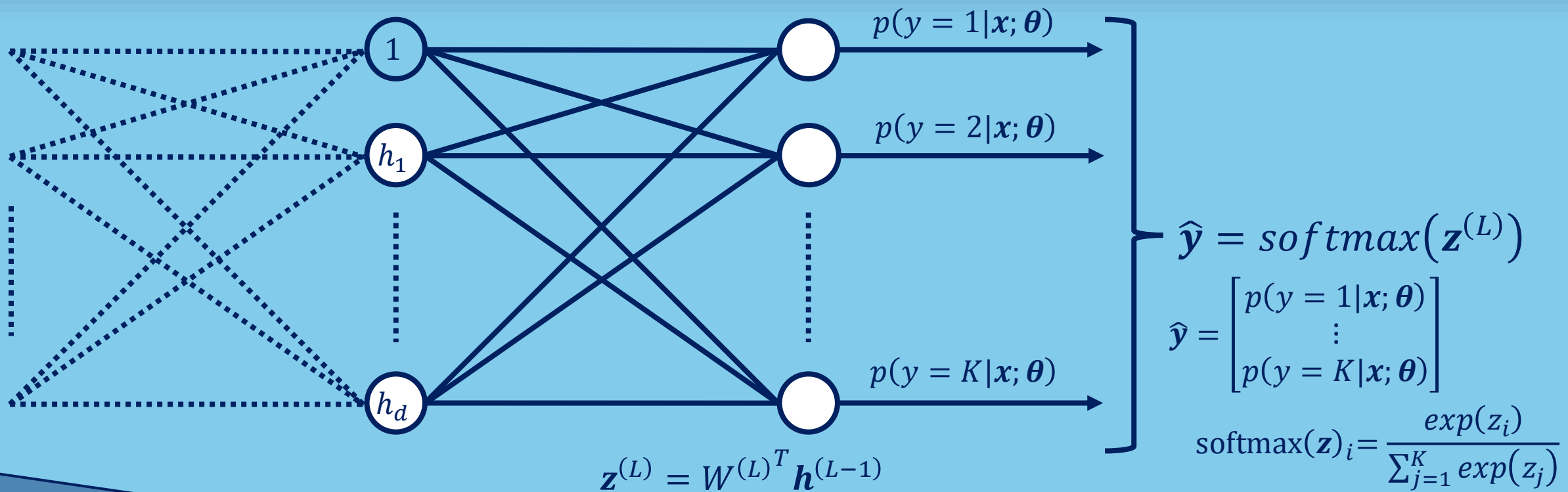
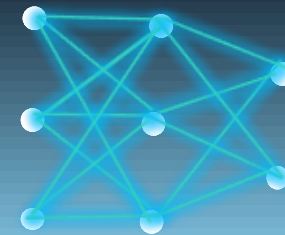
$$J(\boldsymbol{\theta}) = \sum_{n=1}^N \zeta \left((1 - 2y_n) z_n^{(L)} \right)$$

$$\zeta(x) = \log(1 + e^x)$$

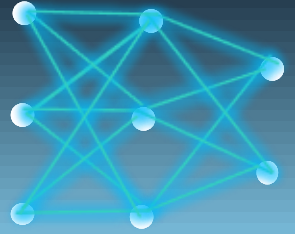
$$\zeta(x) = -\log(\sigma(-x)) \quad \frac{d\zeta(x)}{dx} = \frac{d\log(1 + e^x)}{dx} = \sigma(x)$$



Multiclass Classification



Multiclass Classification – Loss Function

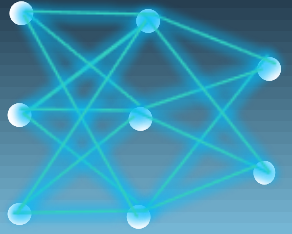


Training Data: $\{(\mathbf{x}_n, y_n)_{n=1}^N\}$ **i.i.d.**

$$J(\boldsymbol{\theta}) = \sum_{n=1}^N \sum_{k=1}^K -y_{nk} \log(\hat{y}_{nk})$$

$$J(\boldsymbol{\theta}) = \sum_{n=1}^N -\log(\hat{y}_{nc})$$

Derivative of Softmax



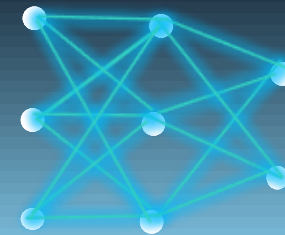
$$p_i = p(y = i | \mathbf{x}; \boldsymbol{\theta}) = \text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)}$$

$$\begin{aligned} \frac{\partial p_i}{\partial z_i} &= \frac{\exp(z_i) \left(\sum_{k=1}^K \exp(z_k) \right) - \left(\exp(z_i) \right)^2}{\left(\sum_{k=1}^K \exp(z_k) \right)^2} = \frac{\exp(z_i) \left(\sum_{k=1}^K \exp(z_k) - \exp(z_i) \right)}{\left(\sum_{k=1}^K \exp(z_k) \right)^2} \\ &= \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \cdot \frac{\sum_{k=1}^K \exp(z_k) - \exp(z_i)}{\sum_{k=1}^K \exp(z_k)} = \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \cdot \left(1 - \frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \right) = p_i(1 - p_i) \end{aligned}$$

$$\frac{\partial p_i}{\partial z_j} = \frac{0 - \exp(z_j) \exp(z_i)}{\left(\sum_{k=1}^K \exp(z_k) \right)^2} = - \left(\frac{\exp(z_j)}{\sum_{k=1}^K \exp(z_k)} \right) \left(\frac{\exp(z_i)}{\sum_{k=1}^K \exp(z_k)} \right) = -p_i p_j, j \neq i$$

$$\frac{\partial p_i}{\partial z_j} = p_i (\delta_{ij} - p_j)$$

Derivative of Cross-Entropy Loss with Softmax

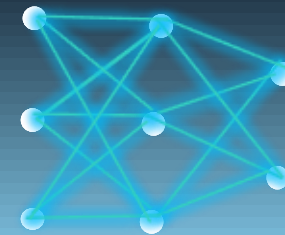


$$J(\boldsymbol{\theta}) = \sum_{n=1}^N \sum_{k=1}^K -y_{nk} \log(p_{nk}) \quad p_{nk} = p(y_n = k | \mathbf{x}_n; \boldsymbol{\theta}) = \text{softmax}(\mathbf{z}_n)_k = \frac{\exp(z_{nk})}{\sum_{j=1}^K \exp(z_{nj})}$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial z_{ni}} = \frac{\partial}{\partial z_{ni}} \left(\sum_{k=1}^K -y_{nk} \log(p_{nk}) \right) = \sum_{k=1}^K -y_{nk} \frac{1}{p_{nk}} \frac{\partial p_{nk}}{\partial z_{ni}}$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial z_{ni}} = \sum_{k=1}^K -y_{nk} \frac{1}{p_{nk}} \cdot p_{nk} (\delta_{ki} - p_{ni}) = \sum_{k=1}^K -y_{nk} (\delta_{ki} - p_{ni}) = -y_{ni} + \left(\sum_{k=1}^K y_{nk} \right) p_{ni}$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial z_{ni}} = (p_{ni} - y_{ni})$$



Thank you